

SEF Academy

Frontend Training Program

A complete 4-week hands-on frontend training program

React.js	Tailwind CSS	REST API
Duration 4 Weeks	Level Intermediate	Projects 2 Projects

2 Projects: Admin Dashboard | Online Store

1. Program Overview

This training program is designed for students who completed the frontend course and built the Ecommerce. The goal now is to learn how to build a professional frontend that communicates with that API.

You will build two real-world projects:

Project 1: Admin Dashboard Design

A full admin control panel for managing the store — products, orders, customers, and analytics.

Project 2: Online Store

A full customer-facing store — registration, shopping, cart, orders, and payment.

Learning Objectives

- Build real React applications that communicate with a REST API
- Use Tailwind CSS to create professional, responsive UIs
- Manage state correctly using `useState`, `useEffect`, and `useContext`
- Handle Authentication (JWT) and protect routes based on role
- Use React Router v6 for client-side navigation
- Provide real-time user feedback (Loading states, Errors, Success toasts)
- Upload images and send `FormData` to the API
- Build charts and analytics views in the Admin Dashboard

Tech Stack

Technology	Purpose	Notes
React.js	Main UI library	v18+ with Hooks
Tailwind CSS	Utility-first styling	Responsive & dark-mode ready
React Router v6	Client-side routing	Protected routes, nested routes
Axios	HTTP requests to API	Interceptors for auth token
React Context API	Global state management	Auth state, Cart state
Recharts	Charts & analytics	Used in Admin Dashboard
React Hook Form	Form management	Validation & error handling
React Toastify	Notifications	Success / Error messages
Vite	Build tool	Fast dev server

2. Project 1 — Admin Dashboard

A complete admin control panel — displays store statistics and manages products, orders, and users. The Admin Dashboard communicates with the /admin endpoints in the API.

2.1 Project Structure

You can handle the project structure that you want

2.2 Pages & Features

Dashboard Page — Home Screen

Displays all store statistics at a glance.

Products Page — Product Management

Full CRUD for products with image uploads. Uses endpoints: GET, POST, PUT, DELETE /products

- Display all products in a table with image, price, and stock
- Search & Filter by category and brand
- Add new product — form with multiple image uploads (multipart/form-data)
- Edit product — same form with ability to delete and add images
- Delete product — confirmation modal before deleting
- Pagination for large product lists

Orders Page — Order Management

View all orders with filtering and status updates. Uses: GET /orders/admin, PATCH /orders/admin/:id/status

- Filter by status (pending, processing, shipped, delivered...)
- Filter by date range (date range picker)
- Order details — expandable row or modal
- Update order status — dropdown select with confirmation
- Color-coded status badges for each order state

Users Page — User Management

View and manage all users. Uses: GET /users/all, POST /users/add, DELETE /users/:id

- List all users with their role and email
- Add a new Admin user
- Delete a user with confirmation dialog
- View user details — orders and saved addresses

2.3 Screens Overview

Screen	Route	Auth Required
Login	/login	Public
Dashboard (Home)	/dashboard	Admin Only
Products List	/dashboard/products	Admin Only
Add Product	/dashboard/products/new	Admin Only
Edit Product	/dashboard/products/:id/edit	Admin Only
Orders List	/dashboard/orders	Admin Only
Order Details	/dashboard/orders/:id	Admin Only
Users List	/dashboard/users	Admin Only
Carts View	/dashboard/carts	Admin Only

2.4 Key Components

Sidebar Navigation

- Fixed sidebar on the left with links to all pages
- Active link highlighting using NavLink
- Collapsible on small screens (mobile drawer)
- User info and Logout button at the bottom

Protected Route Component

- Verifies that the user has a valid token and role = admin
- Redirects to /login if not authenticated as admin
- Fetches user data from the API on first load

Axios Instance

- Base URL from .env (VITE_API_URL)
- Request interceptor automatically adds the Authorization header
- Response interceptor logs out the user on 401 responses

SEF Academy

3. Project 2 — Online Store

A complete customer-facing store — registration, browsing, cart, wishlist, orders, and payment. This project covers the majority of the API endpoints.

3.1 Project Structure

You can handle the project structure that you want

3.2 Pages & Features

Home Page

- Hero section with a CTA button
- Featured Products section — GET /products?featured=true
- Categories section — links to the products page filtered by category
- Newsletter signup (UI only)

Auth Flow (3 Pages)

- Register Page — form that sends POST /auth/register/send-otp
 - Fields: username, email, password, phone
 - On success, redirects to the Verify OTP page
- Verify OTP Page — 6-digit input, sends POST /auth/verify-otp
 - Resend OTP button if the code expires
- Login Page — POST /auth/login, saves the token in localStorage
- Forgot Password — POST /auth/forgot-password/send-otp, then verify

Products Page

- Grid display of products — GET /products with pagination
- Sidebar filters — Category, Brand, Price Range
- Search bar — GET /products/search
- Sort options — price, rating, newest
- ProductCard component — image, name, price, rating, Add to Cart button
- If a product has a discountPrice, show the original price with a strikethrough

Product Detail Page

- Image gallery — displays all product images with thumbnail navigation
- Product info — name, price, description, brand, and category
- Stock indicator — In Stock / Out of Stock
- Add to Cart button — POST /carts/items
- Add to Wishlist button — POST /wishlists/add/:id
- Reviews section — displays ratings with average star score
- Add Review form — POST /products/:id/reviews (logged-in users only)

Cart Page

- Display all cart items — GET /carts
- Quantity update — PATCH /carts/items
- Remove item — DELETE /carts/items/:productId
- Coupon input — POST /carts/coupon with codes (SAVE10, SAVE20...)
- Cart Summary — Subtotal, Discount, Total
- Clear Cart button — DELETE /carts/clear
- Proceed to Checkout button

Wishlist Page

- Display all saved products — GET /wishlists/my
- Remove from wishlist — DELETE /wishlists/remove/:productId
- Move to Cart — adds to cart and removes from wishlist
- Clear all — DELETE /wishlists/clear

Checkout & Payment

- Checkout Page — select shipping address and payment method
 - Cash on delivery
 - Stripe — Card element from Stripe.js
- Order summary — shows items and totals before confirming
- POST /orders to create the order
- If Stripe — renders the Stripe Card Element and completes payment
- Order Success Page after the order is placed

Profile & My Orders

- Profile Page — GET /auth/me, PATCH /users/:id
- Change password and update avatar
- My Orders — GET /orders/my with filter by status
- Order Detail — GET /orders/my/:id

- Cancel Order — PATCH /orders/my/:id/cancel (only if pending or confirmed)

3.3 Screens Overview

Screen	Route	Auth
Home	/	Public
Products	/products	Public
Product Detail	/products/:id	Public
Register	/register	Guest Only
Verify OTP	/verify-otp	Guest Only
Login	/login	Guest Only
Forgot Password	/forgot-password	Guest Only
Cart	/cart	User
Wishlist	/wishlist	User
Checkout	/checkout	User
Payment	/checkout/payment	User
Order Success	/order-success	User
Profile	/profile	User
My Orders	/profile/orders	User
Order Detail	/profile/orders/:id	User

4. 4-Week Training Plan

Each week builds on the previous one. At the end of every week, you submit specific deliverables for review.

Week 1 — Setup & Admin Dashboard Foundation

Topics:

- Initialize both projects (Vite + React + Tailwind CSS setup)
- Axios instance with Auth interceptors
- React Router v6 — Protected Routes for Admin
- AuthContext — Login/Logout state management
- Admin Sidebar & Layout components
- Login Page with form validation
- Dashboard Home Page — Stats cards and summary

Deliverables:

- Admin Dashboard project running with working Login
- Sidebar navigation fully functional
- Dashboard page displaying real data from the API
- Stats cards: Revenue, Orders Count, Customers
- Protected routes blocking non-admin access

Week 2 — Admin Products & Orders Management

Topics:

- Products Page — list products in a table with pagination
- Add Product Form — multipart/form-data with image upload
- Edit Product — update data and delete/add images
- Delete Product with confirmation modal
- Recharts — Revenue Line Chart (last 7 days)
- Orders Page — display and filter orders
- Order Status Update — dropdown with API call

Deliverables:

- Full product CRUD working with Cloudinary image uploads
- Chart displaying daily revenue for the last 7 days
- Orders management with status update
- Users management page
- Top Products table visible on the Dashboard

Week 3 — Online Store Core

Topics:

- Initialize the Online Store project (Vite + Tailwind setup)
- Navbar with Cart icon showing item count
- AuthContext + CartContext for the Store
- Auth Pages — Register, Verify OTP, Login
- Products Page — grid layout with Filters and Search
- Product Detail Page — gallery, info, and reviews
- Cart Page — items, quantity controls, and coupon

Deliverables:

- Full auth flow working (Register -> OTP -> Login)
- Products page with filtering and search
- Product detail page complete with reviews
- Cart page with coupon system
- Wishlist page fully functional

Week 4 — Checkout, Payment & Profile

Topics:

- Checkout Page — shipping address + payment method selection
- Stripe.js Integration — Card element
- Order creation — POST /orders
- Order Success Page
- Profile Page — view and update user data
- My Orders Page with filter and pagination
- Order Detail and Cancel Order
- Final polish — loading states, error handling, responsive design

Deliverables:

- Complete checkout flow working (Cash + Stripe)
- Profile page with update and change password
- My Orders with cancel functionality
- Both projects fully connected to the API
- UI responsive on mobile and desktop

5. Technical Guidelines

5.1 Axios Setup

```
// src/api/axios.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

// Request Interceptor – add token automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

5.2 Sending Images (multipart/form-data)

When uploading images to the Admin Products API, you must send FormData — not JSON:

```
const formData = new FormData();
formData.append('name', productName);
formData.append('price', price);
images.forEach(img => formData.append('images', img));

await api.post('/products', formData, {
  headers: { 'Content-Type': 'multipart/form-data' }
});
```

5.3 State Management Pattern

Use Context API for only 2 global states — Auth and Cart. Keep all other states local inside their components.

State	Location	What it holds
Auth state	AuthContext (Global)	user, token, login(), logout()
Cart count	CartContext (Global)	itemCount, refreshCart()
Products list	Local (useState)	Inside ProductsPage only
Form data	Local (useState)	Inside each Form component

State	Location	What it holds
Loading / Error	Local (useState)	Per API call

5.4 UX Requirements

- Every API call must have a Loading state — spinner or skeleton screen
- Every error must be displayed to the user — not just logged to the console
- Every successful action must show a Toast notification
- Confirmation modal required before any delete action
- Disable buttons while loading to prevent double submissions
- Empty states — if there is no data, show a message and an appropriate illustration
- The UI must work correctly on both mobile and desktop (responsive)

6. Evaluation Criteria

Criterion	Score	Details
API Integration	25%	All endpoints working correctly with proper error handling
UI / UX Quality	20%	Clean design, responsive layout, logical user experience
Auth & Route Protection	15%	JWT, protected routes, role-based access control
State Management	15%	Correct context usage, no unnecessary re-renders
Admin Dashboard	10%	Charts, full CRUD, order management
Checkout & Payment	10%	Stripe integration, complete order flow
Code Quality	5%	Clean code, reusable components, consistent naming

Tips for Success

- Start every feature with the API call before building the UI
- Use console.log frequently at the start, then clean it up before submitting
- Build small reusable components — Button, Input, Modal — from day one
- Name variables and functions clearly and consistently
- Test every feature on both mobile and desktop before submitting

SEF Academy — Frontend Training Documentation

Confidential — Internal Use Only | React.js + Tailwind CSS + Ecommerce API